

System Requirements Specification Document



Naked

Member	Student Number
Arran Lamond	23655675
Byron Pillay	21516261
Keegan Lewis	24594522
Tshepiso Makelana	23588579
Mulondi Makhado	24739163

Table of Contents

Introduction	3
User Stories	3
User	3
Investigator	3
Admin	4
Use Cases	4
Use Case Diagrams	5
Authentication : UC1, UC2, UC3 and UC4	5
Profile Management: UC5 and UC6	5
Browse Cases: UC6 and UC7	6
Report Generation: UC8 and UC9	6
Case Analysis: UC12 and UC13	7
User Management: UC14 and UC15	7
Case Management: UC10, UC11	8
Functional Requirements	8
API Service Contracts	9
EP1: Register	9
EP2: Login	10
EP3: createCase	11
EP4: Fetching Cases	12
EP5: Fetching Single Case	13
EP6: Uploading Evidence	13
Quality Requirements	14
Constraints	14
Technologies Required	15
Architecture Diagram	16
Domain Model	17
Document Details	17

Introduction

Insurance providers, such as our Naked, face a growing challenge from fraudulent AI-generated claims, while also dealing with limited investigative resources in terms of both time and cost. Our product provides a solution to this problem by providing investigators with a centralized workbench for efficient evidence collection and analysis. The platform also generates comprehensive investigative reports and assigns a risk score to each claim, helping investigators prioritize cases and focus their efforts where they are needed most.

User Stories

User

- User: "I want to register an account so I can use the system."
- User: "I want to log in and log out so I can securely access my account."
- User: "I want to reset my password when I forget it to ensure I can retain access to my account"
- User: "I want my settings to be stored and retrieved so that my preferences are preserved between sessions."
- User: "I want to search for a case by case ID, date created, case name, file name, or creator".
- User: "I want to view a case so that I can see its details, evidence, metadata, and status."
- User: "I want a report to include a timeline of events and audit log so I can understand the sequence of activity in the case."
- User: "I want a report to generate a risk score with an explanation for the score, for a case"
- User: "I want the report to include extracted metadata from the evidence so that all relevant technical information is available in one place."
- User: "I want to review a generated report so that I can understand the findings of a submitted case."

Investigator

- Investigator: "I want to create a case so I can begin organising and analysing evidence."
- Investigator: "I want to update cases I created so I can change incorrect or incomplete case information and evidence."
- Investigator: "I want to upload an image, PDF, video, URL link, or audio to a case so it can be included in the case and analysed."
- Investigator: "I want metadata to be extracted from uploaded PDFs, images, audio files, URLs or videos so the file information can be stored with the evidence."
- Investigator: "I want to analyse media side-by-side so I can easily compare evidence files."
- Investigator: "I want to use analysis tools on uploaded media to inspect evidence in more detail."
- Investigator: "I want to submit a case so that it can be analysed and reported on."

- Investigator: "I want to review the generated report so that I can check the findings before finalising it."

Admin

- Admin: "I want to search for users by email or user ID so that I can quickly find a specific user account."
- Admin: "I want to modify a user's access permissions so that I can control what the user is allowed to do in the system."
- Admin: "I want role-based access control to be enforced so that only authorised users can perform restricted actions."

Use Cases

UC1: A User registers an account.

UC2: A User can login.

UC3: A User can logout.

UC4: A User can reset their password.

UC5: A User can update their preferences.

UC6: A User can search for cases by case ID, date created, case name, file name, or creator.

UC7: A User can view a case or report.

UC8: A User can view the audit log and timeline of a case in the report.

UC9: A User can generate a report which contains a risk score and an explanation for the score.

UC10: An Investigator can create, modify and submit cases.

UC11: An Investigator can upload a video, image, PDF, URL or audio to a case.

UC12: An Investigator can analyse media side-by-side.

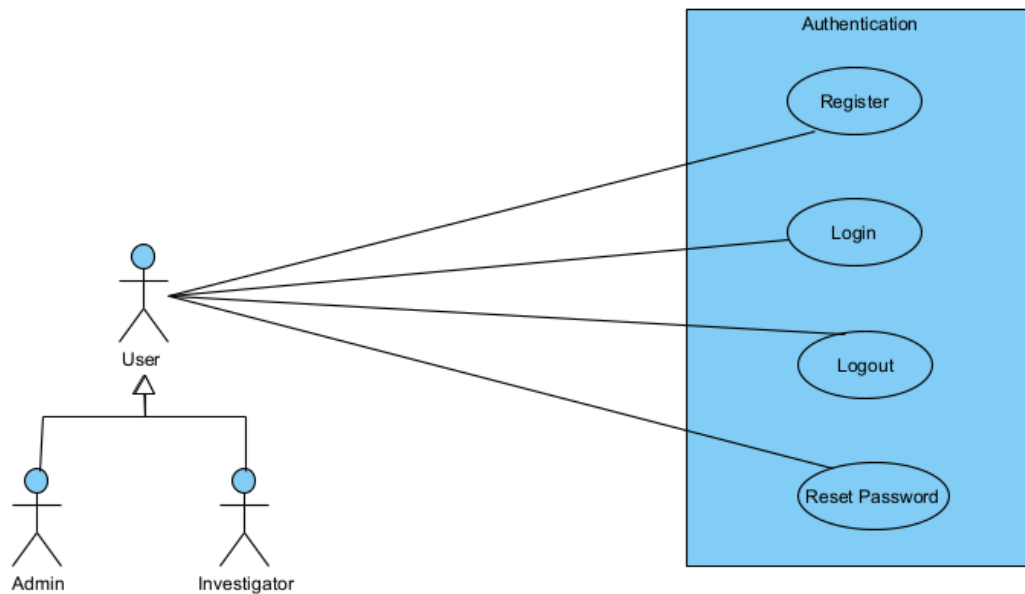
UC13: An Investigator can use the provided analysis tools to analyse the media in the case.

UC14: An Admin can search for users by email or user ID.

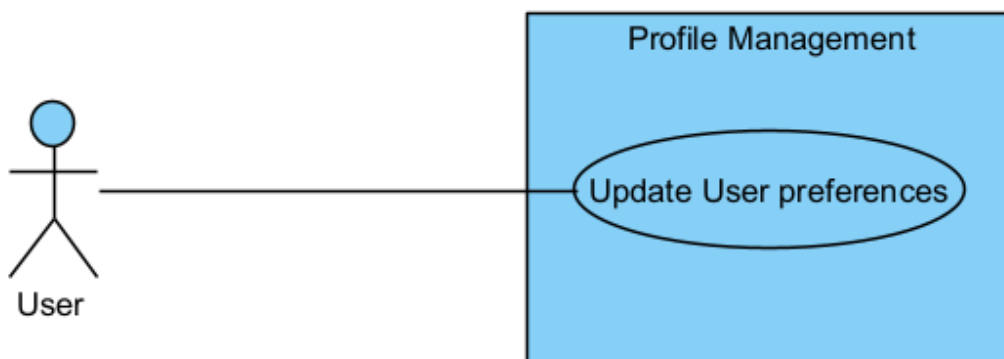
UC15: An Admin can alter the role of a user.

Use Case Diagrams

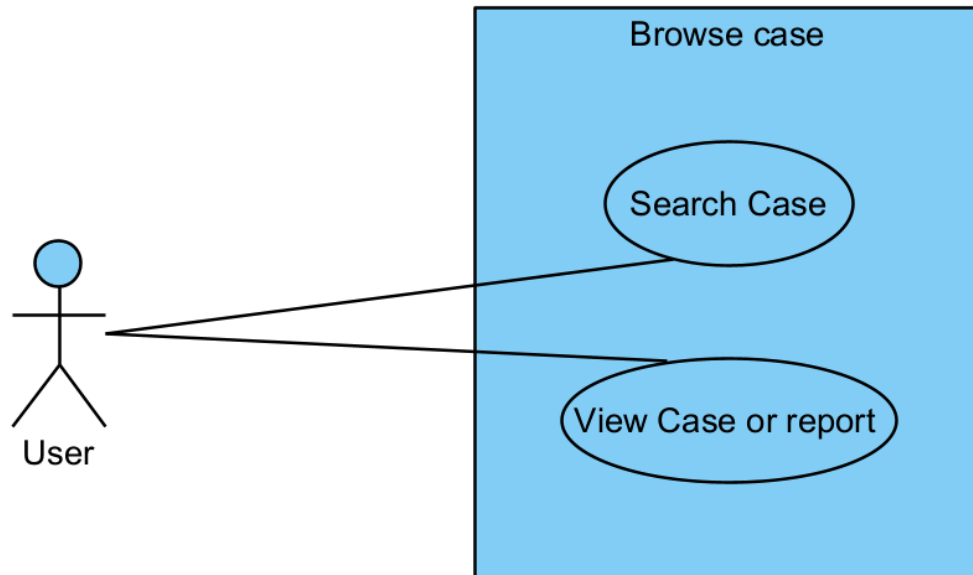
Authentication : UC1, UC2, UC3 and UC4



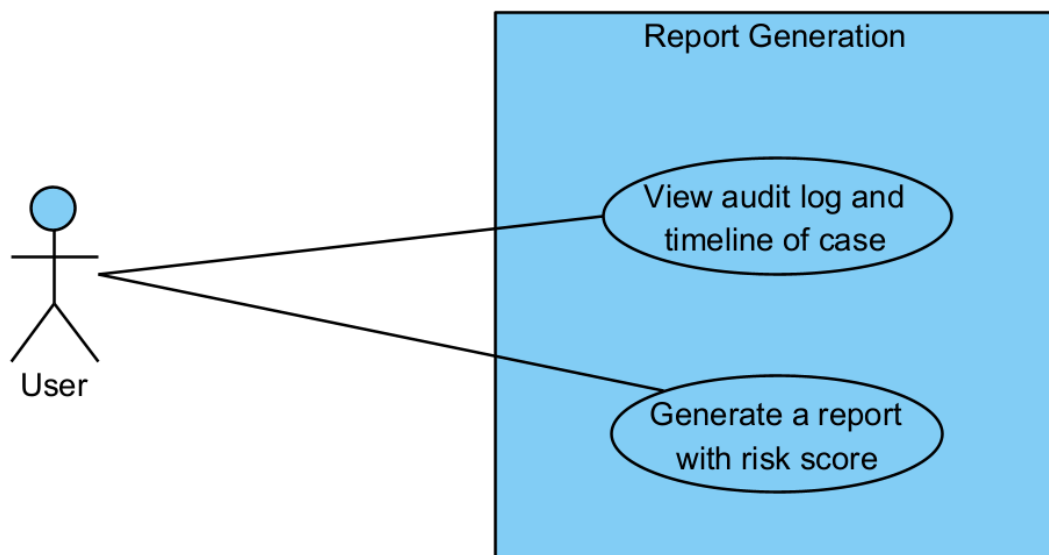
Profile Management: UC5 and UC6



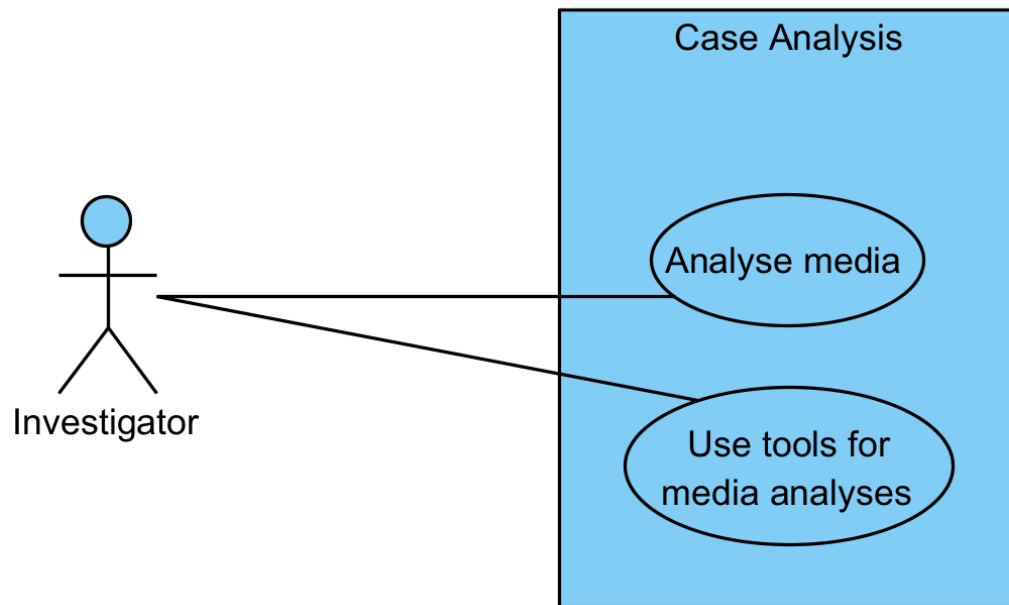
Browse Cases: UC6 and UC7



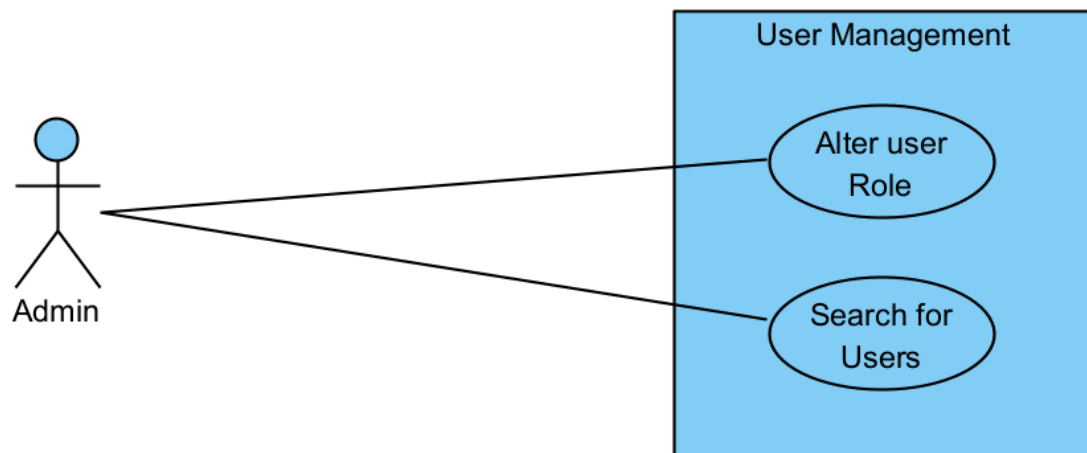
Report Generation: UC8 and UC9



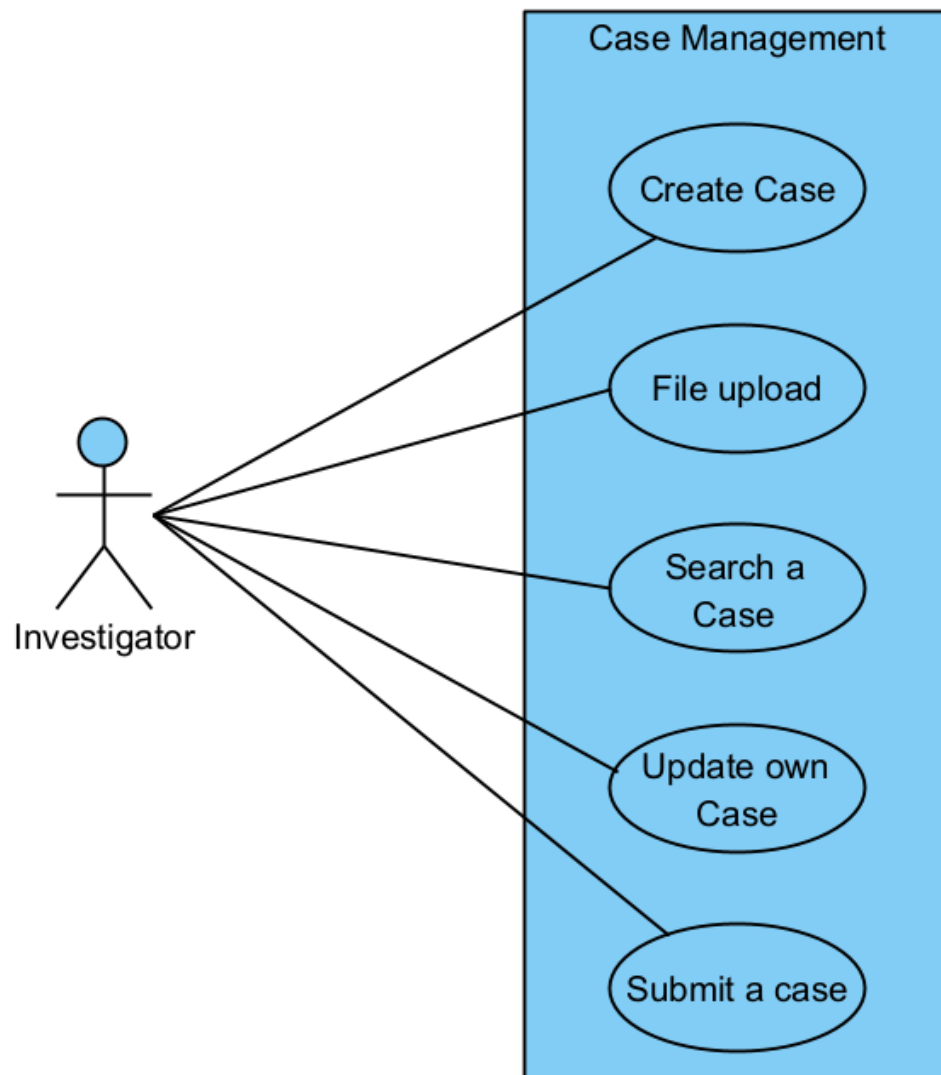
Case Analysis: UC12 and UC13



User Management: UC14 and UC15



Case Management: UC10, UC11



Functional Requirements

R1: User Management and Settings

- R1.1: Authenticate users
- R1.2: Store and retrieve user settings
- R1.3: Allow users to login and logout
- R1.4: Allow users to register
- R1.5: Allow users to reset their password
- R1.6: Allow role based access for admins, investigators and users

R2: Upload Digital Media for a Case

- R2.1: Investigator can upload an image
- R2.2: Investigator can upload a PDF
- R2.3: Investigator can upload an audio file
- R2.4: Investigator can upload a video file
- R2.5: Investigator can link a URL

R3: Manage Cases

- R3.1: An investigator can create a case
- R3.2: The creator of the case can modify the case
- R3.3: An investigator can submit the case

R4: Extract and store metadata

- R4.1: Extract and store the metadata from a PDF
- R4.2: Extract and store the metadata from an image
- R4.3: Extract and store the metadata from an audio file
- R4.3: Extract and store the metadata from a video file
- R4.4: Extract and store the metadata from a linked URL

R5: Searching and Viewing Cases

- R5.1: Users can search for a case by date created, creator and case name.
- R5.2: Users can view a case or report

R6: System-Generated Report on a Submitted Case

- R6.1: Report must include a timeline of events
- R6.2: Report must include an audit log
- R6.3: Report must generate a risk score for the case
- R6.4: Report must include explanations for the score
- R6.5: Report must include the metadata extracted from the evidence

R7: Analyse Media Side-by-side via Analysis Tools

R8: Admin-User Management

- R8.1: An admin can search users via email or user ID
- R8.2: An admin can modify the users access permissions

API Service Contracts

EP1: Register

Endpoint: POST /api/register

Request Headers:

- Content-Type: application/json

Request Body (JSON):

```
{
  "username": "example_username",
  "password": "ExampleP@assword123",
  "email": "example@email.com"
}
```

The following is required of the Request Body:

- The username must be unique.
- The password must meet the following requirements:
 - Must be at least 8 characters long
 - Must include at least one upper case character (A-Z)
 - Must include at least one digit (0-9)

- Must include at least one lower case character (a-z)
 - Must include at least one special character (E.g: !, @, #)
- The email must be unique, and meet the following requirements:
 - At least one of the following characters in any order: 0-9, A-Z, a-z, _, ., + or -
 - An "@" character follows the requirement above
 - At least one of the following characters in any order: 0-9, a-z, A-Z, . or -
 - A "." follows the requirement above
 - At least two of the following characters in any order: A-Z or a-z

Success Response (201):

```
{
  "status": "success",
  "message": "Account created successfully"
}
```

Example Error Response (400):

```
{
  "status": "error",
  "message": "Invalid or missing username"
}
```

Possible Error Response Codes:

- 400 - Returned when there are invalid or missing fields in the request body
- 409 - The email or username in the request body is not unique (Account already exists)

EP2: Login

Endpoint: POST /api/login

Request Headers:

- Content-Type: application/json

Request Body (JSON):

```
{
  "username": "example_username",
  "password": "ExampleP@assword123"
}
```

The following is required of the Request Body:

- The password must meet the following requirements:
 - Must be at least 8 characters long
 - Must include at least one upper case character (A-Z)
 - Must include at least one digit (0-9)
 - Must include at least one lower case character (a-z)
 - Must include at least one special character (E.g: !, @, #)
- The email must be unique, and meet the following requirements:
 - At least one of the following characters in any order: 0-9, A-Z, a-z, _, ., + or -
 - An "@" character follows the requirement above

- At least one of the following characters in any order: 0-9, a-z, A-z, . or -
- A "." follows the requirement above
- At least two of the following characters in any order: A-Z or a-z

Success Response (200):

```
{
  "status" : "success",
  "token" : "Example_token_here"
}
```

Example Error Response (401):

```
{
  "status" : "error",
  "message" : "Invalid or missing username"
}
```

Possible Error Response Codes:

- 400 - Returned when the password or email in the request body does not meet requirements
- 401 - Returned when the password is incorrect
- 404 - Returned when the user does not exist
- 409 - The email or username in the request body is not unique (Account already exists)

EP3: createCase

Endpoint: POST /api/createCase

Request Headers:

- Content-Type: application/json
- Authorization: Bearer <token_here>

Request Body (JSON):

```
{
  "title" : "Example title here",
  "description" : "Example description of the case here."
}
```

The following is required of the Request Body:

- The title cannot be blank or missing.

Success Response (201):

```
{
  "status" : "success",
  "CaseId" : "Example_case_id_here"
}
```

Example Error Response (403):

```
{
```

```
    "status" : "error",
    "message" : "User unauthorized"
}
```

Possible Error Response Codes:

- 400 - Returned when the request body does not meet the requirements
- 401 - Returned when there is a JWT related error
- 403 - Returned when the user is not authorized to access the endpoint

EP4: Fetching Cases

Endpoint: POST /api/getCases

Request Headers:

- Content-Type: application/json
- Authorization: Bearer <token_here>

Request Body (JSON):

```
{
}
```

Success Response (201):

```
{
  "status": "success",
  "cases": [
    {
      "CaseId": "Example id here",
      "CaseCreator": "Creator here",
      "CaseName": "Suspicious AI-generated claim",
      "CaseReviews": null,
      "CaseDescription": "Example description",
      "CaseClosed": false,
      "CaseCreationDate": "2026-05-21T11:30:00"
    }
  ]
}
```

Example Error Response (401):

```
{
  "status" : "error",
  "message" : "JWT related error here"
}
```

Possible Error Response Codes:

- 401 - Returned when there is a JWT related error

EP5: Fetching Single Case

Endpoint: POST /api/getCases

Request Headers:

- Content-Type: application/json
- Authorization: Bearer <token_here>

Request Body (JSON):

```
{
  "caseID" : "Some case ID"
}
```

The following is required of the Request Body:

- The caseID must not be missing or empty, and it must be a valid UUID.

Success Response (200):

```
{
  "status": "success",
  "case": {
    "CaseId": "Example id here",
    "CaseCreator": "Creator here",
    "CaseName": "Suspicious AI-generated claim",
    "CaseReviews": null,
    "CaseDescription": "Example description",
    "CaseClosed": false,
    "CaseCreationDate": "2026-05-21T11:30:00"
  }
}
```

Example Error Response (404):

```
{
  "status" : "error",
  "message" : "JW"
}
```

Possible Error Response Codes:

- 401 - Returned when the CaseID is not a valid UUID
- 400 - Returned when the CaseID is missing
- 404 - Returned when the case does not exist

EP6: Uploading Evidence

Endpoint: POST /api/evidence

Request Headers:

- Content-Type: multipart/form-data
- Authorization: Bearer <token_here>

Request Body (JSON):

```
{
  "caseID" : "Some case ID",
  "media" : "uploaded file"
}
```

The following is required of the Request Body:

- The caseID must not be missing or empty, and it must be a valid UUID,
- The media file must be uploaded.

Success Response (201):

```
{
  "status": "success",
  "evidence": {
    "MediaId": "Example media UUID here",
    "url": "http://localhost:9000/example-bucket/example-file.png",
    "Status": "uploaded"
  }
}
```

Example Error Response (404):

```
{
  "status": "error",
  "message": "Case not found"
}
```

Possible Error Response Codes:

- 400 - Returned when the file extension is unsupported or invalid caseid
- 401 - Returned when there are jwt or uuid errors
- 403 - Returned when the user is unauthorized
- 404 - Returned when the case does not exist
- 422 - Returned by FastAPI when the case media is missing from the form

Quality Requirements

QR1: The system must respond to normal user requests within 2 seconds.

QR2: The system must protect user accounts using JWT-based authentication.

QR3: The system must only allow authorized users to access case evidence.

QR4: The system must store uploaded media reliably without losing files.

QR5: The system must support common evidence file types such as images (E.g: .jpg, .png) and PDFs.

QR6: The system must be usable through a clear and consistent user interface.

QR7: Unit tests must be completed for the developed features before entering dev.

Constraints

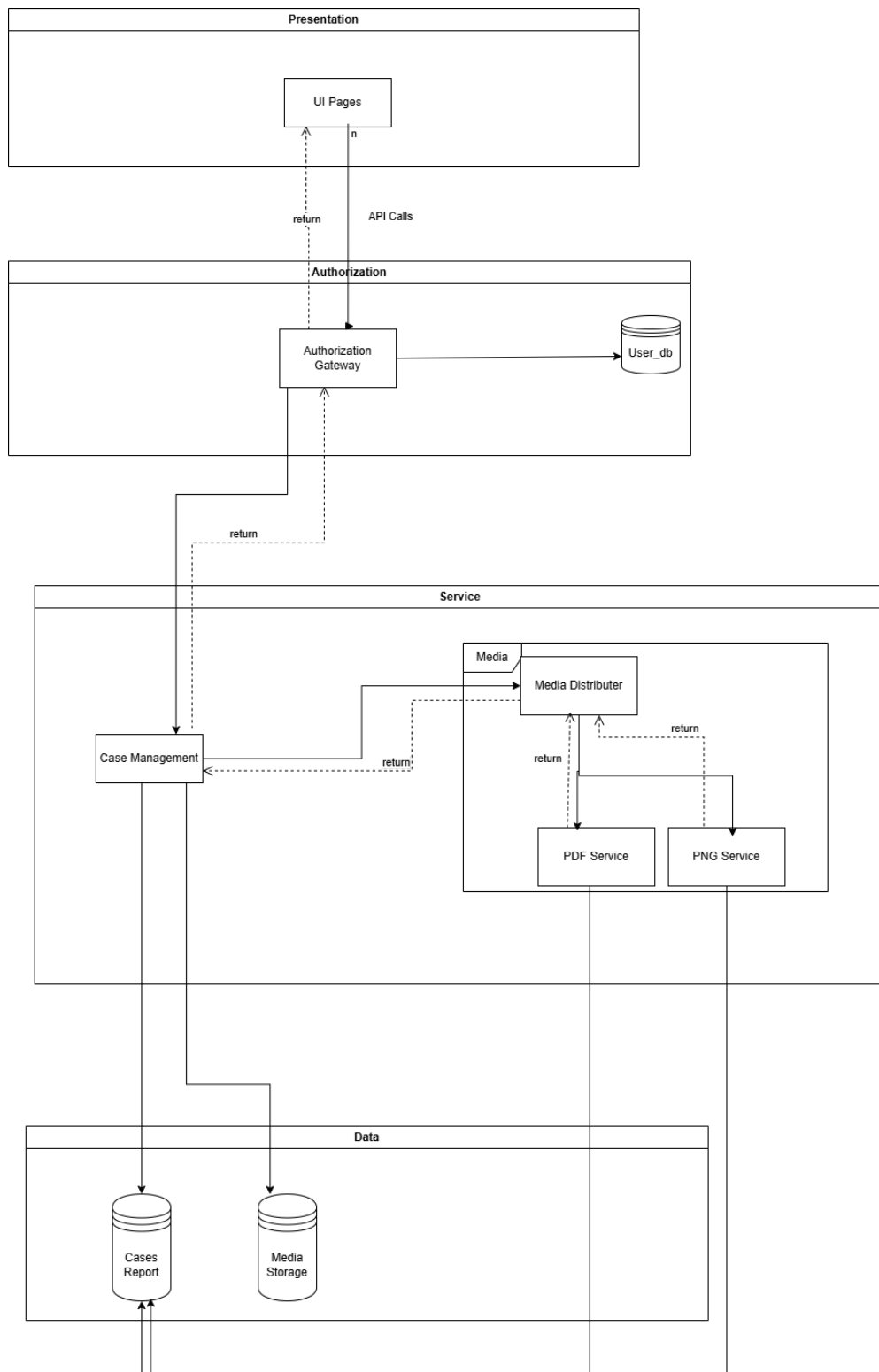
The system should be developed with free, open source technologies.

Technologies Required

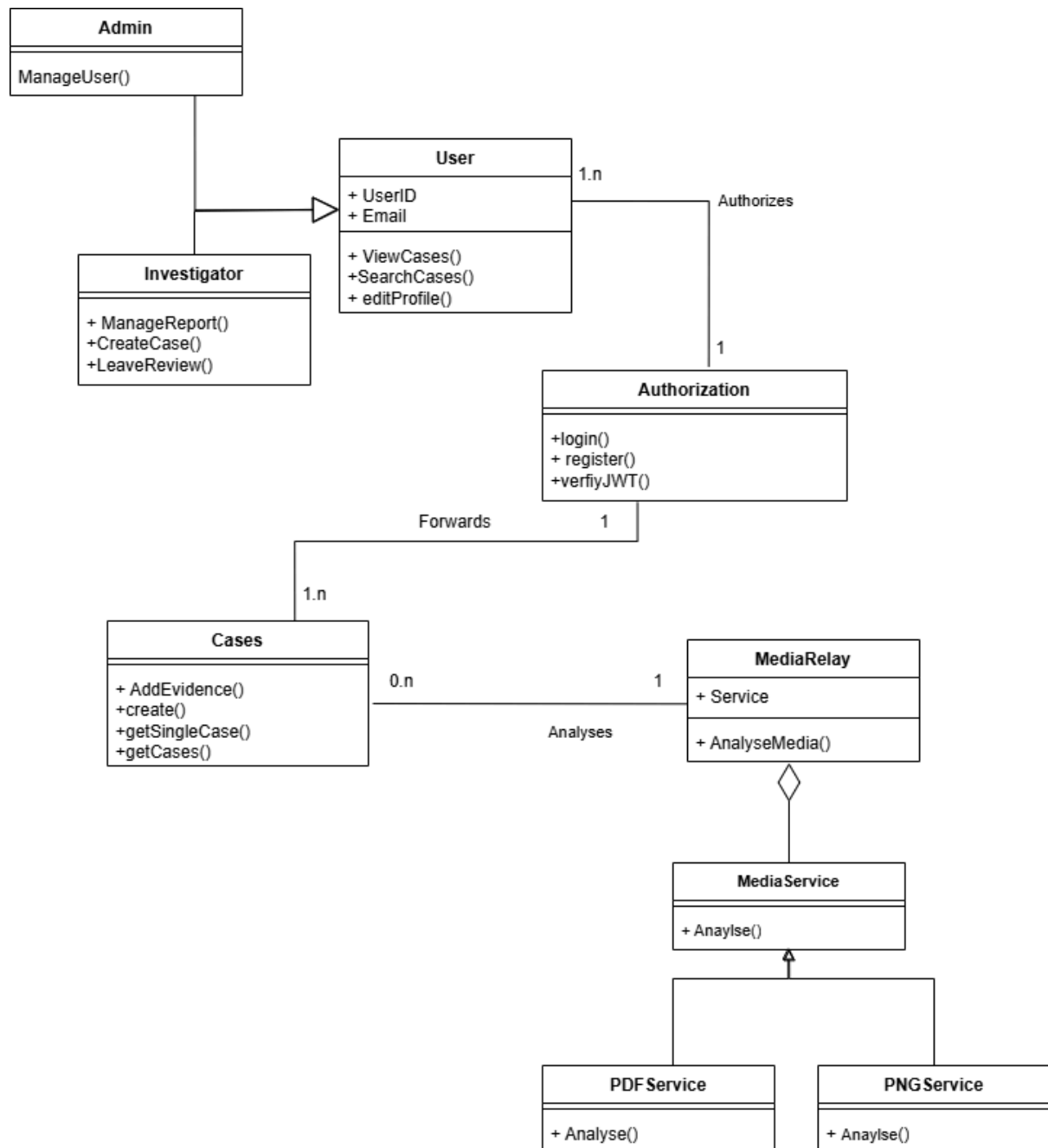
- Next.js
- Python
- FastAPI
- OpenSearch
- PostgreSQL
- MINIO
- Pytorch, OpenCV, Hugging Face
- Redis
- Celery
- Docker
- Playwright
- GitHub

Note: Any packages required by the system can be installed by following the instructions in the provided README file, found in the repo.

Architecture Diagram



Domain Model



Document Details

Version	1.3
Date Modified	21/05/2026
Details	-Added the Admin user and created the relationship analyse - Specified the function in Authorization

	better, added more methods to the case and removed the attributes from Cases - Added the list of technologies required by the system. - Modified the Use Cases and structured the order of the document. - Added Quality Requirements. - Added Service Contracts and the following endpoints: login, register, getCases, getSingleCase, and evidence
--	---